

Simulación 1

INSTITUTO: **Instituto de Tecnología e Ingeniería**

CARRERA/S: **Licenciatura en Desarrollo de Videojuegos y Simulaciones**

RESPONSABLE DE LA ASIGNATURA Y EQUIPO DOCENTE:
Facundo Saiegh / José Guerra Prado

AÑO: 2026

CRÉDITOS: 7 créditos

CARGA HORARIA DE INTERACCIÓN PEDAGÓGICA: 64 horas

CARGA HORARIA TOTAL: 175 horas

CÓDIGO DE LA MATERIA EN SIU:

1. Fundamentación

La simulación constituye una de las principales herramientas para el estudio, análisis y modelización de sistemas complejos. En el ámbito de la informática y del desarrollo de videojuegos, permite representar fenómenos físicos, biológicos, sociales y artificiales mediante modelos computacionales, facilitando la experimentación, la validación de hipótesis y el análisis de diferentes escenarios sin necesidad de intervenir sobre el sistema real.

En la industria de los videojuegos, las técnicas de simulación son utilizadas para construir mundos virtuales, modelar el comportamiento de objetos y agentes, generar contenido procedural, representar fenómenos naturales, optimizar procesos y desarrollar sistemas interactivos capaces de responder de manera coherente a las acciones del usuario. Asimismo, estas herramientas poseen aplicaciones en áreas como la ingeniería, la robótica, la inteligencia artificial, la visualización científica y la simulación de procesos.

La asignatura propone introducir a los estudiantes en los fundamentos de la simulación computacional desde una perspectiva eminentemente práctica, utilizando el desarrollo de videojuegos como principal contexto de aplicación. Se abordarán tanto modelos determinísticos como estocásticos, simulaciones discretas y continuas, así como diferentes paradigmas de modelado utilizados actualmente en motores de videojuegos y sistemas interactivos.

A lo largo de la cursada se desarrollarán implementaciones propias de los algoritmos estudiados, privilegiando la comprensión de sus fundamentos, sus limitaciones, su costo computacional y las técnicas de optimización necesarias para ejecutar simulaciones en tiempo real. Se promoverá además el análisis crítico de distintas estrategias de modelado, permitiendo a los estudiantes seleccionar las herramientas más adecuadas según el problema a resolver.

La materia constituye un primer acercamiento a la simulación computacional y proporciona las bases conceptuales necesarias para asignaturas posteriores relacionadas con inteligencia artificial, física computacional, motores de videojuegos, gráficos por computadora y desarrollo de simuladores

2. Propósitos y/u objetivos

Al finalizar la asignatura se espera que los estudiantes sean capaces de:

- Modelar sistemas mediante simulaciones computacionales.
- Diferenciar modelos discretos, continuos, determinísticos y estocásticos.
- Implementar algoritmos de generación procedural.
- Desarrollar simulaciones físicas y sistemas basados en partículas y grillas.
- Implementar simulaciones mediante autómatas celulares y otros modelos de vida artificial.
- Modelar sistemas utilizando simulación de eventos discretos.
- Analizar algoritmos mediante notación Big O y estimar su escalabilidad.
- Diseñar y ejecutar benchmarks para evaluar el rendimiento de diferentes

implementaciones.

- Aplicar el método científico para formular hipótesis, diseñar experimentos, analizar resultados y validar modelos de simulación.
- Optimizar simulaciones mediante técnicas de organización de datos, estructuras eficientes y análisis de rendimiento.
- Aplicar los conocimientos adquiridos al desarrollo de videojuegos, simuladores y sistemas interactivos.

3. Contenidos mínimos

Modelos de simulación. Simulación como modelización de sistemas. Clasificación de modelos. Etapas en el desarrollo de un proyecto de simulación. Validación, experimentación y optimización. Modelos de simulación discretos y continuos. Modelos matemáticos y algoritmos. Simulación estocástica y determinística. Aleatoriedad.

4. Carga horaria

Créditos	Interacción pedagógica	Trabajo autónomo	TOTAL
7	64	111	175

5. Programa analítico

Unidad 1: Mundos procedurales 2d y 3d

Números pseudoaleatorios con semilla (seeded random) y funciones hash como generadores deterministas de bajo costo de memoria. Funciones de ruido coherente (Perlin, Simplex, Value Noise, Worley) y sus parámetros de configuración. Funciones de suavizado e interpolación (lerp, smoothstep/sigmoid) aplicadas a la generación de ruido y terreno. Diagramas de Voronoi aplicados a la partición de territorios y mapas estilizados. Generación de mapas a partir de campos de ruido y manejo de memoria mediante chunks. Combinación de múltiples campos de ruido (temperatura, humedad, altura, pendiente) para la definición de biomas mediante reglas compuestas. Herramientas de autoría: generadores de mapas, autotiling y generación procedural de caminos.

Unidad 2: Falling sand y Simulación de Fluidos

El modelo de autómatas celulares aplicado a materiales granulares (arena, agua, fuego) con renderizado directo por píxel. Simulación en mundos de gran escala o infinitos

mediante particionado del espacio. Integración con cuerpos rígidos (rigid bodies). Integración con campos de flujo (flow fields) para aproximar comportamiento fluido. Enfoque basado en grilla / celdas (píxeles). Enfoque basado en partículas: fundamentos y motores de referencia (Liquid Fun). Smoothed Particle Hydrodynamics (SPH) y reconstrucción de superficie mediante marching squares. Método híbrido PIC/FLIP. Simulación de fluidos por campo de altura (height-field / 1.5D).

Unidad 3: Vida artificial y autómatas celulares Optimización de simulaciones

Autómatas celulares clásicos: el Juego de la Vida y su formalización mediante convolución de kernels. Extensiones del modelo: vecindarios múltiples (Multiple Neighborhoods Cellular Automata). Transición al dominio continuo: SmoothLife. Modelos continuos de vida artificial: Lenia y Flow Lenia como generalización matemática de los autómatas celulares. Sistemas de partículas con reglas de interacción emergente: Particle Life y Particle Lenia. Análisis de complejidad algorítmica (notación Big O) aplicado a simulaciones. Organización de datos: Array of Structures vs. Structure of Arrays (SoA) y diseño orientado a datos (data-oriented design). Impacto del acceso a memoria: cache misses y comportamiento del garbage collector. El método científico aplicado a la optimización: formulación de una hipótesis de rendimiento (por ejemplo, "convertir esta estructura a SoA reduce el tiempo de frame"), diseño del experimento, medición mediante benchmarks y comparación cuantitativa entre una implementación baseline y la implementación modificada. Benchmarking comparativo (por ejemplo, el Juego de la Vida implementado con paradigma orientado a objetos versus SoA) como caso de aplicación del método anterior. Alternativas de ejecución de alto rendimiento en el navegador: WebAssembly. Caso de estudio de optimización sobre un proyecto propio de la cátedra (WeedJS).

Unidad 4: Ray casting y Terreno destructible

Concepto de ray casting: proyección de un rayo y detección de la primera intersección con la escena. Resolución matemática de intersecciones rayo-superficie. Algoritmo DDA (Digital Differential Analyzer) sobre grillas, optimizado mediante spatial hashing. Aplicación histórica de referencia: el renderizado pseudo-3D de Wolfenstein 3D. Cálculo de iluminación mediante ray casting. Autómatas celulares clásicos: el Juego de la Vida y su formalización mediante convolución de kernels. Extensiones del modelo: vecindarios múltiples (Multiple Neighborhoods Cellular Automata). Transición al dominio continuo: SmoothLife. Modelos continuos de vida artificial: Lenia y Flow Lenia como generalización matemática de los autómatas celulares. Sistemas de partículas con reglas de interacción emergente: Particle Life y Particle Lenia.

Unidad 5: Simulación de sonido

Fundamentos del audio digital: frecuencia de muestreo y profundidad de bits como discretización de una señal continua. Filtros digitales (pasa bajos, pasa altos, pasa

banda). Espacialización sonora: sonido estéreo y sonido 3D. Procesamiento en tiempo real con Web Audio API (AudioContext, AudioWorklet) y automatización de parámetros.

Unidad 6: Simulación de eventos discretos y método Montecarlo

Simulación de eventos discretos (DES): modelado de sistemas de colas (por ejemplo, cajas de un supermercado). Estructuras de datos de soporte: heap, min-heap, heapify, heap sort y sistemas de cola de eventos. Método Montecarlo como herramienta de experimentación estocástica. Caso de aplicación integrador: testeo de estrategias de blackjack mediante simulación estocástica repetida, como validación experimental de un modelo.

Unidad 7: Motores de física: cuerpos rígidos con Box2D

Concepto de cuerpo rígido (rigid body) como unidad básica de un motor de físicas. Fundamentos de la física de Newton aplicados a la simulación: fuerzas, masa, velocidad angular. Métodos de integración numérica: dinámica basada en impulsos (impulse-based) frente a dinámica basada en posición (position-based), e integración de Verlet. Detección de colisiones: teorema del eje separador (SAT), volúmenes envolventes (AABB, OBB) y jerarquías de volúmenes envolventes (BVH). Dinámica angular: velocidad angular, angular drag, linear drag. Restricciones y articulaciones (constraints / joints): weld, distance, pin, mouse. Optimización del solver: coloreo de grafos de restricciones (graph coloring) y agrupamiento de cuerpos en reposo (islands).

6. Bibliografía y recursos

6.1. Bibliografía obligatoria

- Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). Discrete-event system simulation (5th ed.). Pearson.
- Millington, I. (2010). Game physics engine development: How to build a robust commercial-grade physics engine for your game (2nd ed.). CRC Press.
- Nystrom, R. (2014). Game programming patterns. Genever Benning.
- Shiffman, D. (2024). The nature of code: Simulating natural systems with JavaScript. No Starch Press. <https://natureofcode.com/>
- Sommerville, I. (2011). Ingeniería de software (9a ed.). Addison-Wesley.

6.2. Bibliografía optativa

- Bridson, R. (2015). Fluid simulation for computer graphics (2nd ed.). CRC Press.

- Chan, B. W. C. (2020). Lenia: Continuous-space cellular automata. *Complex Systems*, 28(2), 125–155. <https://doi.org/10.25088/ComplexSystems.28.2.125>
- Ericson, C. (2004). *Real-time collision detection*. Morgan Kaufmann.
- Fabian, R. (2018). *Data-oriented design: A crash course in efficiency*. Richard Fabian. <http://www.dataorienteddesign.com/dodmain/>
- Farnell, A. (2010). *Designing sound*. MIT Press.
- Short, T. X., & Adams, T. (Eds.). (2017). *Procedural generation in game design*. CRC Press.

6.3. Recursos

- Kenney.nl (Asset Jesus): Repositorio libre de tilesets 2D/3D, spritesheets de partículas, vectores y sonidos con licencia CC0 para pruebas de generación procedural y físicas. <https://kenney.nl/>
- OpenGameArt.org: Banco colaborativo de recursos gráficos y de audio de código abierto. <https://opengameart.org/>
- Lode's Computer Graphics Tutorial (Raycasting): Tutorial clásico con derivación matemática y código del algoritmo DDA y renderizado pseudo-3D al estilo Wolfenstein 3D. <https://lodev.org/cgltutor/raycasting.html>
- Lenia & Continuous Cellular Automata World: Repositorio e interfaz interactiva para la exploración de autómatas celulares continuos (Lenia, Flow Lenia, SmoothLife). <https://chakaz.github.io/>
- Particle Life Simulation Demos: Visualizadores en tiempo real en WebGL para la simulación de vida artificial basada en atracción/repulsión de partículas. <https://particle-life.com/>
- Redblobgames (Amit Patel): Guías visuales e interactivas sobre algoritmos de mapas (Voronoi, ruido de Perlin, grids de ruinas y pathfinding). <https://www.redblobgames.com/>
- Box2D / LiquidFun Documentation: Documentación del motor de físicas 2D de cuerpos rígidos e integración de fluidos por partículas. <https://box2d.org/>
- MDN Web Audio API Docs: Guía oficial y ejemplos de procesamiento de sonido en tiempo real, filtros e integración de AudioWorklet. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- Aula Virtual UNAHUR (Moodle): Espacio de consultas asincrónicas y videos tutoriales de corta duración para asistencia en la configuración de la interfaz.

7. Metodología de enseñanza

La materia se desarrollará mediante clases expositivas y explicativas, promoviendo en todo momento la participación activa del alumnado. Se fomentará la apertura de espacios de debate y reflexión sobre los contenidos abordados, con el objetivo de fortalecer el pensamiento crítico y la comprensión profunda de los conceptos.

Al inicio del desarrollo de cada unidad temática, uno o dos grupos de estudiantes

realizarán una breve exposición introductoria sobre los conceptos centrales del tema, en base a una investigación previa realizada por su cuenta, previo a que el/la docente desarrolle la unidad en profundidad.

Se realizarán ejercicios orales y escritos que funcionarán como repaso, práctica y aplicación de los temas tratados. Además, se incentivará la investigación y búsqueda de información por parte de los estudiantes, en relación con los distintos tópicos de la asignatura, promoviendo el desarrollo de habilidades autónomas y colaborativas.

La evaluación será continua, encuentro a encuentro, considerando la participación, la resolución de ejercicios, el análisis de casos y la integración de contenidos entre unidades. Se trabajará especialmente en la articulación de conceptos teóricos con situaciones prácticas propias del desarrollo de software.

Como eje integrador de la cursada, los alumnos llevarán adelante un proyecto grupal de prototipo de videojuego, simulando el funcionamiento de un estudio independiente. Este proyecto se desarrollará a lo largo de todo el cuatrimestre, aplicando los contenidos de cada unidad, y será presentado públicamente durante las dos últimas clases.

7.1. Trabajo autónomo de la/el estudiante

Actividad de Trabajo Autónomo	Carga horaria estimada
Investigación de documentación, foros de soporte y análisis comparativo de licencias	30 horas
Desarrollo práctico de los prototipos lúdicos dentro del motor (diseño y físicas)	51 horas
Pruebas de optimización, control de errores y exportación final	30 horas

8. Evaluación y régimen de aprobación

*Por su carácter de materia taller, con un fuerte componente de seguimiento, experimentación en laboratorio e integración práctica incremental de software, esta asignatura **no contempla** la posibilidad de rendir bajo la condición de estudiante libre.*

8.1. Modalidad de evaluación

La evaluación de la materia se organizará en tres componentes principales: trabajos prácticos, evaluaciones parciales y criterios de calidad. El enfoque busca promover el aprendizaje progresivo, la creatividad y el desarrollo de propuestas sólidas, respetando

buenas prácticas de trabajo.

- **Trabajos prácticos:** Durante la cursada se solicitará la entrega de trabajos prácticos vinculados al desarrollo de proyectos y metodologías de trabajo. Cada trabajo podrá ser reentregado en caso de que sea necesario mejorar aspectos señalados en la devolución. Las reentregas serán evaluadas teniendo en cuenta la evolución del trabajo, la incorporación de sugerencias y el compromiso con el proceso de mejora.

Este componente representa el 40% de la nota final.

- **Evaluaciones Parciales:** Durante la cursada se realizarán dos evaluaciones parciales:

Primer parcial: evaluación teórico-práctica sobre contenidos conceptuales fundamentales en formato escrito.

Segundo parcial: entrega de un proyecto completo que incluya documentación básica y armado de tablero de proyecto. La presentación y defensa oral será obligatoria.

Cada parcial tendrá su correspondiente instancia de recuperación.

Este componente representa el 50% de la nota final (25% cada parcial).

Criterios de Evaluación

Todos los trabajos y evaluaciones se corregirán considerando los siguientes criterios:

- Claridad: exposición coherente de ideas, uso adecuado del lenguaje y organización.
- Diseño: creatividad, funcionalidad y consistencia del proyecto.
- Buenas prácticas: aplicación de principios éticos, metodológicos y técnicos.

Estos criterios se aplicarán como ajuste cualitativo sobre el promedio, representando el 10% restante de la nota final.

8.2. Aprobación de la cursada

Para aprobar la cursada y obtener la condición de regular, el régimen académico establece que debe obtenerse una nota no inferior a cuatro (4) puntos. Todas las instancias evaluativas tienen una instancia de recuperatorio. Podrán acceder a la administración de esta modalidad solo aquellos y aquellas estudiantes que hayan obtenido una nota inferior o igual a 6 (seis) puntos en el examen parcial.

Siempre que se realice una evaluación de carácter recuperatorio, la calificación que los/as estudiantes obtengan reemplazará la calificación obtenida en el examen que se ha recuperado y será la considerada definitiva a los efectos de la aprobación.

Los/las estudiantes deben poseer una asistencia no inferior al 75% (setenta y cinco por ciento) en las clases presenciales. En cuanto a la cursada de materias virtuales se requerirá que el/la estudiante ingrese al aula virtual de la materia como mínimo una vez por semana. Este requisito se mantendrá en aquellas carreras que se desarrollen bajo la modalidad a distancia.

8.3. *Acreditación de la materia*

La materia puede aprobarse por promoción, evaluación integradora o examen final.

Promoción directa: tal como lo establece el Régimen Académico, para acceder a esta modalidad, el/la estudiante deberá aprobar la cursada de la materia con una nota no inferior a siete (7) puntos, no obteniendo en ninguna de las instancias de evaluación parcial menos de seis (6) puntos, sean evaluaciones parciales o recuperatorios. El promedio estricto resultante deberá ser una nota igual o superior a siete (7) sin mediar ningún redondeo.

Evaluación integradora: tal como lo establece el Régimen Académico, podrán acceder a esta evaluación aquellos estudiantes que hayan aprobado la cursada con una nota de entre cuatro (4) y seis (6) puntos.

La evaluación integradora tendrá lugar por única vez en el primer llamado a exámenes finales posterior al término de la cursada. Deberá tener lugar en el mismo día y horario de la cursada y será administrado, preferentemente, por el/la docente a cargo de la comisión. Se aprobará tal instancia con una nota igual o superior a cuatro (4) puntos, significando la aprobación de la materia. La nota obtenida se promediará con la nota de la cursada.

Examen final: Instancia destinada a quienes opten por no rendir la evaluación integradora o hayan regularizado la materia en cuatrimestres anteriores. Se evalúa la totalidad de los contenidos del programa de la materia y se aprueba con una calificación igual o superior a cuatro (4) puntos. Esta nota no se promedia con la cursada.